

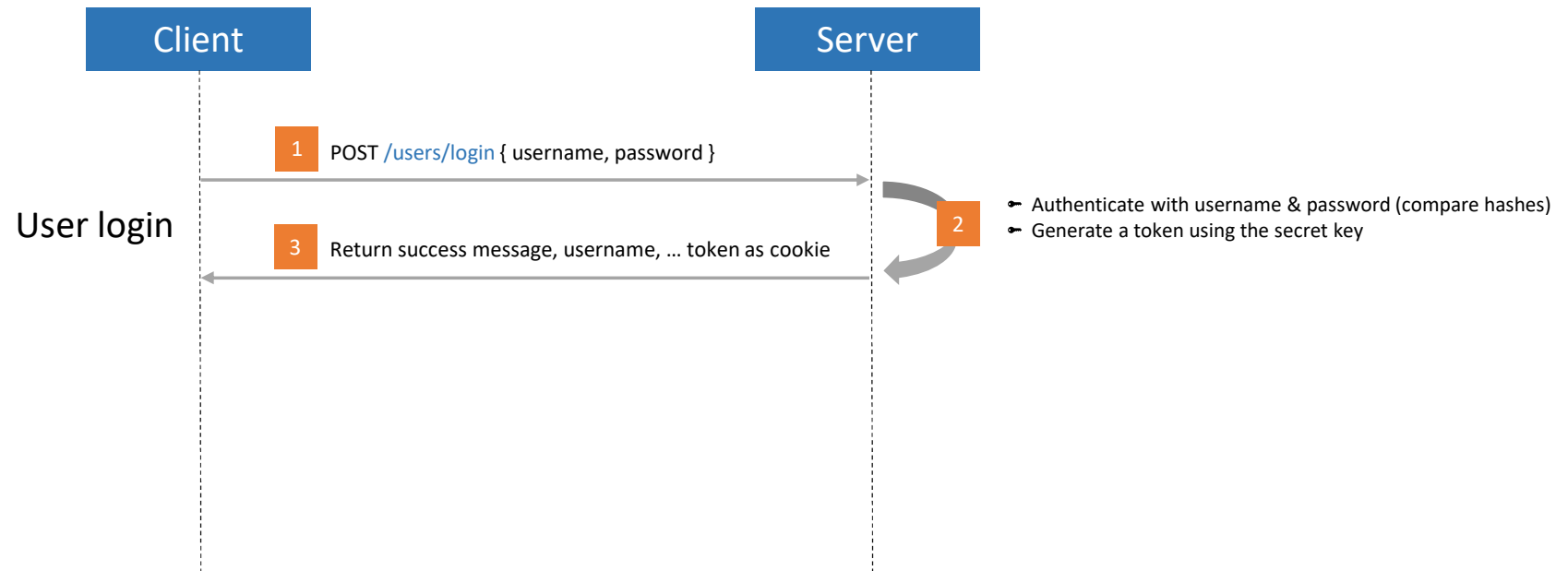


Full-stack software development

Front-end security

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

Recap: Authentication



Front-end security

- Implement login functionality in the front-end.
- Handle secure HttpOnly cookies sent from the back-end.
- Manage user session data in the UI.
- Adapt the UI based on a user's role.

Courses App

[Home](#)[Lecturers](#)[Schedules](#)[Login](#)

Login succesful. Redirecting to homepage...

johanp

.....

[Login](#)

The screenshot shows the 'Response' tab selected in the browser's developer tools. The main pane displays the JSON body of the response:

```
{
  "message": "Authentication successful.",
  "token": null,
  "username": "johanp",
  "fullname": "Johan Pieck",
  "role": "lecturer"
}
```

Below this, the 'Headers' section is expanded, showing the 'Response Headers'. A checkbox labeled 'Raw' is present. The headers are listed as follows:

Header	Value
Access-Control-Allow-Credentials	true
Access-Control-Allow-Origin	http://localhost:8080
Access-Control-Expose-Headers	Set-Cookie
Cache-Control	no-cache, no-store, max-age=0, must-revalidate
Connection	keep-alive
Content-Type	application/json
Date	Sun, 02 Nov 2025 05:41:08 GMT
Expires	0
Keep-Alive	timeout=60
Pragma	no-cache
Set-Cookie	authToken=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpzcmMiOjBjb3Vyc2VzX2FwcCklbnN1Yil6ImpvaGFucCklsmV4cCl6MTZc2MjA5MDg2OCwiaWF0IjoxNzYyMDYyMDY4LCJzY29wZSI6ImUPTeVFtEVDFVFSRVlfiQ.71O1zXvIG2S_MmWichMlyZKHesjKJsgM6wTf3Nx30xl; Path=/; Max-Age=3600; Expires=Sun, 02 Nov 2025 06:41:08 GMT; Secure; HttpOnly; SameSite=None

After successful login, the back-end response contains the user data in JSON and sets a secure cookie containing the JWT token via the Set-Cookie header.

Submitting login form

```
const handleSubmit = async (event: { preventDefault: () => void }) => {
  event.preventDefault();
  clearErrors();

  if (!validate()) {
    return;
  }

  const user = { username: name, password };

  try {
    const loggedInUser = await UserService.authenticate(user);
    setStatusMessages([
      { message: t('success'), type: 'success' }
    ]);
    sessionStorage.setItem('loggedInUser', JSON.stringify(userData));

    setTimeout(() => {
      router.push('/');
    }, 2000);
  } catch (error) {
    setStatusMessages([
      {
        message: (error as Error).message || tGeneral('error.unknownError'),
        type: 'error',
      },
    ]);
  }
};
```

- Convert JSON response object (containing username and role) to string and store in session storage for later use.
- SessionStorage is only available in client components for persisting data in the browser storage.

UserService authenticate

```
const authenticate = async (user: User): Promise<User> => {  
  const response = await fetch(  
    `${process.env.NEXT_PUBLIC_API_URL}/users/login`,  
    {  
      method: 'POST',  
      headers: {  
        'Content-Type': 'application/json',  
      },  
      body: JSON.stringify(user),  
      credentials: 'include',  
    }  
  );  
  
  if (!response.ok) {  
    const error = await response.json().catch(() => ({}));  
    throw new Error(error?.message || `Authentication failed.`);  
  }  
  
  return response.json();  
};
```

- Instruct the browser to accept and safely store the Set-Cookie header.
- The Set-Cookie header contains the JWT token that we need for subsequent API requests.

Access protected pages



Login

Lecturers

You are not authorized to view this page. Please login first.

Courses App			
Home	Lecturers	Schedules	Logout
Welcome, Johan Pieck!		Language	English ▼
Lecturers			
Firstname	Lastname	E-mail	Expertise
Johan	Pieck	johan.pieck@ucll.be	Full-stack development, Front-end development
Jeroen	Rombouts	jeroen.rombouts@ucll.be	Software Engineering, Back-End Development
Bram	Van Impe	bram.vanimpe@ucll.be	Full-Stack development, Back-end Development
Ruben	Naudts	ruben.naudts@ucll.be	Full-Stack development, Front-end Development

Access protected pages in client components

```
const authenticate = async (user: User): Promise<User> => {  
  const response = await fetch(  
    `${process.env.NEXT_PUBLIC_API_URL}/users/login`,  
    {  
      method: 'POST',  
      credentials: 'include',  
    }  
  );  
  return response.json();  
};
```

×	Headers	Preview	Response	Initiator	Timing	Cookies
▶ General						
▶ Response Headers (10)						
▼ Request Headers ☐ Raw						
Accept	/*					
Accept-Encoding	gzip, deflate, br, zstd					
Accept-Language	nl,en;q=0.9,en-GB;q=0.8,en-US;q=0.7,nl-BE;q=0.6					
Connection	keep-alive					
Cookie	NEXT_LOCALE=en; authToken=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpzZW50L3VzZXZlbnQ6IjE1Yil6ImpvaGFucCIsImV4cCI6MTc2MjA5MDg2OCwiaWF0IjoxNzYyMDYyMDY4LCJzY29wZSI6IjE1PTEVfTEVDVfVSRVliifQ.71O1zXvIG2S_MmWlchMIyZKHesjKJsgM6wTf3Nx30xl; __next_hmr_refresh_hash__=5390f14b2668016e31bb6a9cabcfb8f0c8b50fc12ae61a6d					

Instructs the browser to read the JWT token from cookie storage and send it as a “Cookie” request header.

Access protected pages: server components

Remember that server components are rendered on the server.

⇒ No access to browser cookie storage.

⇒ We need to manually send the JWT token in an **Authorization** header

The java back-end supports both scenario's: authentication using an Authorization header or secure cookies.

Access protected pages: server components

```
const getData = async (): Promise<DataResult> => {
  try {
    const cookieStore = await cookies();
    const lecturers = await LecturerService.getAllLecturers(cookieStore);

    return { data: lecturers, error: null };
  } catch (error) {
    return { data: null, error: (error as Error).message };
  }
};

const LecturersPage = async () => {
  const { data, error } = await getData();

  return (
    <>
      <h1>Lecturers</h1>
      {error && (
        <div className="text-red-800" role="alert">
          {error}
        </div>
      )}

      {data && <LecturersClientPage initialLecturers={data} />}
    </>
  );
};
```

- `cookies()`: a **server-side** function from Next.js to read the incoming request cookies it just received from the browser.
- We pass these cookies to the service.
- The service is now able to manually add an Authorization header.

Access protected pages: server components

```
const getAllLecturers = async ( cookies: ReadonlyRequestCookies) => {  
  const token = cookies.get('authToken')?.value;  
  
  const response = await fetch(`${process.env.NEXT_PUBLIC_API_URL}/lecturers`, {  
    method: 'GET',  
    headers: {  
      'Content-Type': 'application/json',  
      Authorization: `Bearer ${token}`,  
    },  
    cache: 'no-store',  
    credentials: 'include',  
  });  
  
  return response.json();  
};
```

- Read the authToken from de cookieStore.
- Add the Authorization header for server components.
- Include secure cookie for client components.

Manage user session data in UI

- Authentication works, but not all UI components are “aware” of this. E.g., Header now needs to show the user name and a logout link.
- Problem: how do we get user data from UserLoginForm to Header?
- Answer: **React Context**
 - We need to create a “AuthContext” component
 - ... that serves as a container for storing user data.
 - While we’re at it, we can also provide helper methods like “login” and “logout”.

Why not with sessionStorage?

- No **Reactivity**: sessionStorage doesn't update the UI. If the data changes, components using that data won't get notified.
- **React Context = Live state**: changes in context trigger re-render. All components in the UI that use the global state (context) will be re-rendered.
- Responsibilities:
 - Use state (and context) for **reactivity**
 - Use sessionStorage for **persistence** (e.g. between browser sessions)

AuthContext

- Manages the user state: hold the visible user data (like name and role) in React's useState.
- Provides a login function: save the user data (received from the login page) into the React state and sessionStorage.
- Provides a logout function: call the UserService.logout (to clear the HttpOnly cookie on the server) and simultaneously clears the user state from React and sessionStorage.
- Restores state on refresh: use useEffect to check sessionStorage on page load, ensuring the user remains logged in on the front-end after a refresh.
- Shares data globally: use a Context.Provider to make the user, login, and logout functions available to any component in the application.

```
type AuthContextType = {
  user: User | null;
  login: (userData: User) => void;
  logout: () => void;
};

export const AuthContext = createContext<AuthContextType | undefined>(
  undefined
);

export const AuthProvider = ({ children }: { children: ReactNode }) => {
  const [user, setUser] = useState<User | null>(null);

  useEffect(() => {
    const storedUser = sessionStorage.getItem('loggedInUser');
    if (storedUser) {
      setUser(JSON.parse(storedUser));
    }
  }, []);

  const login = (userData: User) => {
    sessionStorage.setItem('loggedInUser', JSON.stringify(userData));
    setUser(userData);
  };

  const logout = async () => {
    try {
      await UserService.logout();
    } catch (error) {
      console.error('Logout failed:', error);
    }
    sessionStorage.removeItem('loggedInUser');
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
};
```

Provide AuthContext in RootLayout

```
const RootLayout = async (...) => {  
  ...  
  
  return (  
    <html lang={locale}>  
      <body>  
        <AuthProvider>  
          <Header />  
          <main className="...">  
            {children}  
          </main>  
        </AuthProvider>  
      </body>  
    </html>  
  );  
};  
  
export default RootLayout;
```

- AuthProvider wraps the entire application.
- Makes global authentication state (user data, login, logout) available to every component.

Fill AuthContext in UserLoginForm

```
export const UserLoginForm = () => {  
  ...  
  
  const context = useContext(AuthContext);  
  if (!context)  
    throw new Error('UserLoginForm must be used within an AuthProvider');  
  const { login } = context;  
  
  const handleSubmit = async (e: React.FormEvent<HTMLFormElement>) => {  
  
    e.preventDefault();  
    try {  
  
      const loggedInUser = await UserService.authenticate({ username, password });  
  
      login(loggedInUser);  
  
      router.push('/');  
    } catch (error) { /* ... */ }  
  };  
};
```

Retrieve the login function from AuthContext

Fill AuthContext with the authenticated user, so it will be stored in sessionStorage

Consume AuthContext in Header

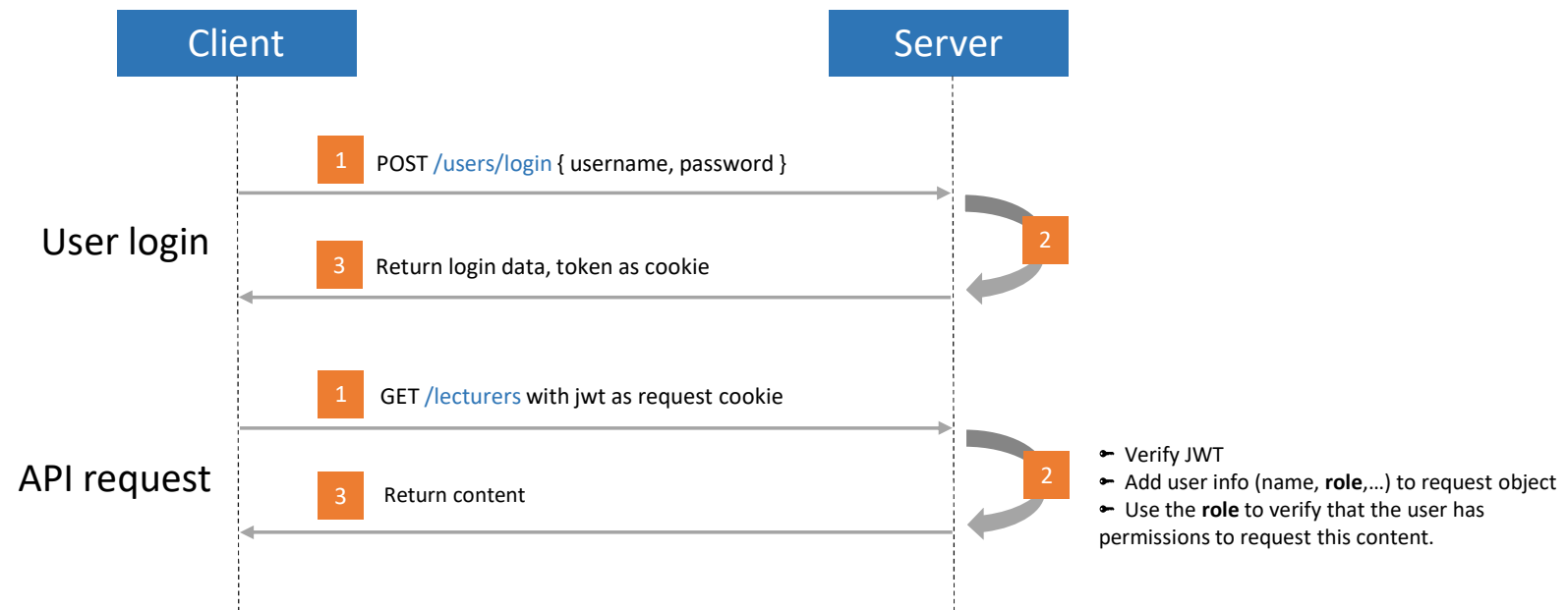
```
export const Header = () => {  
  const context = useContext(AuthContext);  
  if (!context) return <header>...</header>;  
  const { user, logout } = context;  
  
  return (  
    <header>  
      {/* ... (nav links) ... */}  
      <div>  
        {user ? (  
          <span>Welcome, {user.fullname}</span>  
          <button onClick={logout}>Logout</button>  
        ) : (  
          <Link href="/login">Login</Link>  
        )}  
      </div>  
    </header>  
  );  
};
```

Retrieve user data and the logout function from AuthContext

Check if there is an authenticated user in AuthContext

Display the user data from AuthContext, bind the onClick event to the logout function

Authorisation



Return (more) data based on role

Courses App				
Home	Lecturers	Schedules	Logout	Welcome, Johan Pieck!

Schedule for Johan Pieck				
Course	Start	End	Lecturer	Enrolled students
Full-stack development	11-11-2023 08:30	11-11-2023 10:30	Johan Pieck	1

Students		
Firstname	Lastname	Studentnumber
Peter	Parker	r0785023
Bruce	Banner	r0785024
Sally	Smith	r0785025
Michael	Miller	r0785026
Linda	Lawson	r0785027

Role “lecturer”: just see your own schedule

Courses App				
Home	Lecturers	Schedules	Logout	Welcome, admin admin!

Schedule for all users (admin)				
Course	Start	End	Lecturer	Enrolled students
Full-stack development	11-11-2023 08:30	11-11-2023 10:30	Johan Pieck	1
Full-stack development	11-11-2023 13:30	11-11-2023 15:30	Elke Steegmans	1
Software Engineering	11-11-2023 13:30	11-11-2023 15:30	Elke Steegmans	1
Back-end Development	11-11-2023 10:45	11-11-2023 12:45	Greetje Jongen	1

Students		
Firstname	Lastname	Studentnumber
Peter	Parker	r0785023
Bruce	Banner	r0785024
Sally	Smith	r0785025
Michael	Miller	r0785026
Linda	Lawson	r0785027

Role “admin”: see schedules of all lecturers
Back-end returns other data based on role.

Enable features based on role

Courses App			
Home	Lecturers	Schedules	Logout
Welcome, Johan Pieck!			
Lecturers			
Firstname	Lastname	E-mail	Expertise
Johan	Pieck	johan.pieck@ucll.be	Full-stack development, Front-end development
Elke	Steegmans	elke.steegmans@ucll.be	Software Engineering, Back-end Development
Greetje	Jongen	greetjejongen@ucll.be	Full-stack development, Back-end Development

Courses taught by Elke			
Name	Description	Phase	Credits
Full-stack development	Learn how to build a full stack web application.	2	6
Software Engineering	Learn how to build and deploy a software application.	2	6

Role “lecturer”: just see the courses

Courses App			
Home	Lecturers	Schedules	Logout
Welcome, admin admin!			
Lecturers			
Firstname	Lastname	E-mail	Expertise
Johan	Pieck	johan.pieck@ucll.be	Full-stack development, Front-end development
Elke	Steegmans	elke.steegmans@ucll.be	Software Engineering, Back-end Development
Greetje	Jongen	greetjejongen@ucll.be	Full-stack development, Back-end Development

Courses taught by Elke			
Name	Description	Phase	Credits
Full-stack development	Learn how to build a full stack web application.	2	6
Software Engineering	Learn how to build and deploy a software application.	2	6

Role “admin”: allow the scheduling of a course

Use “role” from AuthContext

```
export const CourseOverviewTable = ({ courses, lecturer }: Props) => {  
  
  const context = useContext(AuthContext);  
  const user = context?.user;  
  
  return (  
    <table>  
      <tbody>  
        {courses.map((course) => (  
          <tr key={course.id}>  
            <td>  
  
              {user && user.role === 'admin' && (  
                <Link href={` /schedule/${lecturer.id}/${course.id}`}>  
                  <button>Schedule</button>  
                </Link>  
              )}  
            </td>  
          </tr>  
        ))}  
      </tbody>  
    </table>  
  );  
};
```

- Consume AuthContext. We only need user data.
- Check the user's role and conditionally render the schedule button.

Logout in AuthContext

```
const logout = async () => {  
  try {  
    await UserService.logout();  
  } catch (error) {  
    console.error('Logout failed:', error);  
  }  
  
  sessionStorage.removeItem('loggedInUser');  
  setUser(null);  
};
```

- Call the back-end via UserService: cookies will be cleared.
- Clear browser sessionStorage.
- Clear AuthContext user state.

Recap logout flow: Why call the backend?

```
@PostMapping("/logout")
public ResponseEntity<Object> logout(HttpServletResponse response) {
    ResponseCookie cookie = ResponseCookie.from("authToken", "")
        .httpOnly(true)
        .secure(true)
        .path("/")
        .maxAge(0)
        .sameSite(SameSiteCookies.NONE.toString())
        .build();
    response.addHeader(HttpHeaders.SET_COOKIE, cookie.toString());
    return ResponseEntity.ok(Map.of("message", "Logout successful"));
}
```

- HttpOnly cookies are not accessible via client-side JavaScript. This prevents XSS attacks but also means we cannot delete them directly in the browser.
- Therefore, we must call the API (/logout) to instruct the server to end the session.
- The server responds with a Set-Cookie header containing Max-Age=0, forcing the browser to delete the cookie immediately.